

Policy Platform

Product requirements and technical design for the control plane through which regulators, networks, risk, product, platforms, PayFacs, franchises and merchants all express constraints — without any of them building a rules engine. First delivery: regulatory blocking by BIN, BIN range, currency and geography.

Version	1.0
Status	For review
Companion to	Spec v3.6 §06
First use case	Regulatory blocking

Contents

A0	Summary and the ask
A1	The problem
A2	Users and jobs
A3	Scope
A4	Functional requirements
A5	Non-functional requirements
A6	Success measures
B1	Where it sits
B2	Namespaces and identity
B3	Capability catalogue
B4	Attribute vocabulary
B5	Policy object and data model
B6	Effects, algebra, precedence
B7	Exceptions
B8	Compilation and conflict detection
B9	Distribution and runtime
B10	APIs
B11	Lifecycle and failure modes
C1	First use case — why this one
C2	The four policy shapes
C3	BIN matching mechanics
C4	Stored credentials and mandates
C5	Boundary with screening
C6	Acceptance tests
D1	Disposition of the reviewed approach

Summary and the ask

Nine constituencies need to constrain payment behaviour. Today each either waits for an engineering release or builds a private mechanism. The Policy Platform gives all of them one authoring surface, one precedence model, one audit trail and one explanation — and gives the payment path a compiled artefact rather than a database to query.

Policy determines what is permitted. Decisioning determines what is optimal. Orchestration executes the result.

Ask	Detail
Build the Policy Platform as a first-class platform product	Alongside Payment, Decisioning, Credential, Processing, Money and Intelligence — not as a component of any one of them
Deliver regulatory blocking first	BIN, BIN range, currency and geography. Deterministic, high value, no optimisation surface, and it exercises every part of the platform
Name a constraint authority owner	Open question 14 in the architecture specification. The platform cannot ship without one

The problem

Today	Consequence
A regulatory block requires an engineering change	Weeks between a rule taking effect and the platform honouring it. The gap is an exposure, and it is not currently measurable
Each team builds its own rule mechanism	Risk has one, routing has one, onboarding has one. None share precedence, none share audit, and none can explain the others
Merchant configuration lives in product-specific tables	"Why did this transaction decline" requires someone who knows which table to look in
No conflict detection	A platform prohibition and a merchant permission coexist until a transaction discovers them
No simulation	The blast radius of a rule change is discovered in production
No record of what was in force	A regulatory finding cannot be answered with evidence, only with reconstruction

Users and jobs

User	Job to be done	Today
Compliance	Make a regulatory rule effective on a date, prove it was in force, and show which transactions it touched	Engineering ticket, then trust
Network relations	Encode a scheme mandate before its deadline	Engineering ticket
Risk	Stop an attack pattern within minutes, then withdraw the block cleanly	Private rule engine, no shared audit
Product	Define when a capability is available and what it requires	Code
Merchant	Express their own constraints and see whether they had any effect	Support ticket, and often no answer
Platform / PayFac / ISV	Constrain an estate of submerchants, and delegate part of that authority downward	Not possible
Franchise	Constrain stores and terminals it does not own operationally	Not possible
Operations	Answer "why did this decline" in minutes without an engineer	Escalation
Solutions and sales	Answer "can this merchant do X" before promising it	Ask engineering

Scope

In scope	Out of scope
Authoring, versioning, approval, compilation, distribution, evaluation, explanation of policy	Optimisation. Policy defines the feasible space; the decision platform chooses within it
All nine authoring constituencies	Risk model inference. Risk produces predictions; a policy may act on them
Capability catalogue and attribute vocabulary	Entitlement management. Policy constrains; entitlement grants. Separate planes, separate owners
Simulation, conflict detection, exceptions	Merchant preference management, which is a different plane with different override semantics
Emergency propagation	Sanctions and watchlist screening, which is party screening and remains its own module

The boundary that will be argued about

Policy and entitlement look similar and are not. Entitlement answers "has this client bought and been approved for this capability"; policy answers "given that they have it, is this particular transaction permitted to use it". They have different owners (commercial versus compliance), different change velocities and different override semantics. Merging them produces a single table where a sales concession and a regulatory prohibition have equal standing, which is the failure this platform exists to prevent.

Functional requirements

#	Requirement	Priority
F1	Author a policy in a shared DSL, scoped by namespace, with a declared owner	P0
F2	Target effects at capabilities from a governed catalogue, not at arbitrary services	P0
F3	Reference attributes from a governed vocabulary; unknown attributes fail compilation	P0
F4	Version every policy, with approval recorded and an immutable checksum	P0
F5	Record both validity time and knowledge time on every version	P0
F6	Detect conflicts at compile time and name the resolution	P0
F7	Compile to a signed bundle; distribute to the data plane; evaluate in process	P0
F8	Emit a provenance trace on every decision: which policy, which version, which authority	P0
F9	Answer "can subject S use capability C in context X" outside the payment path	P0
F10	Simulate a proposed change against historical traffic and report exact decision changes	P1
F11	Explain a decision in terms a support agent can relay to a merchant	P0
F12	Express exceptions with reason, owner, approval and mandatory expiry	P1
F13	Delegate bounded authority downward, with recorded grants	P1
F14	Propagate an emergency prohibition within seconds, fail-closed	P0
F15	Report a policy that never fires as inert	P1
F16	Shadow and canary a bundle before activation	P1
F17	Roll back to a prior bundle version by pointer change	P0
F18	Answer "which transactions were decided under a rule we later corrected"	P0

Non-functional requirements

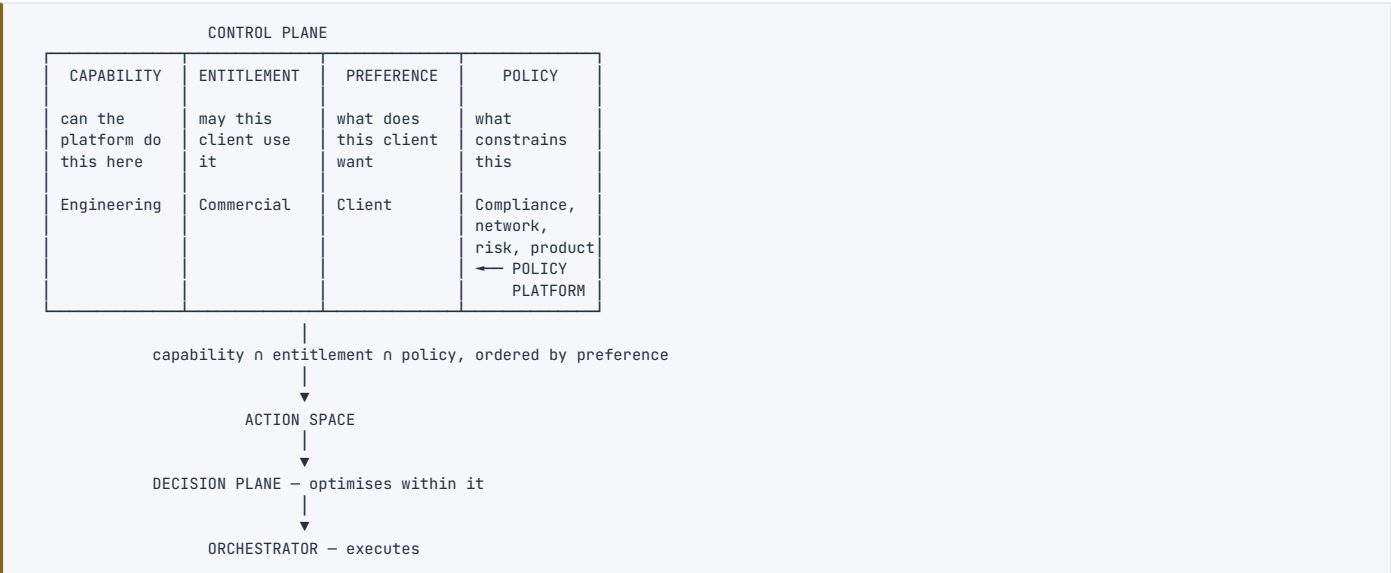
#	Requirement	Target
N1	Evaluation latency on the authorization path	p50 1.0 ms, p99 3.0 ms — in process, no network call
N2	Normal bundle propagation, globally	p99 60 s
N3	Emergency prohibition propagation	p99 5 s, fail-closed on unconfirmed nodes
N4	Control plane unavailability	Data plane continues on held bundles until TTL; degrades to most restrictive held set thereafter
N5	Bundle integrity	Signed; invalid signature rejected and prior version retained
N6	Audit retention	Full version history and decision provenance for the regulatory retention period
N7	Data residency	Bundles compiled and distributed per residency zone; evaluation never leaves the zone
N8	Authoring throughput	A compliance author can take a rule from draft to shadow without engineering involvement

Success measures

Measure	Why it is the right one
Time from rule issued to rule in force	The core value. Weeks to hours is the claim
Share of constraints expressed as policy rather than code	Measures whether the platform is actually being adopted or routed around
Mean time to explain a policy-driven decline	Leading indicator for support cost and for merchant trust
Conflicts caught at compile time versus in production	Direct measure of whether the compiler is earning its keep
Inert policy count	Rules that never fire mean the configuration surface is lying to someone
Exception count and age	Exceptions are the sprawl vector; they are owned by product, not by whoever requested them
Emergency channel usage rate	Routine use means the normal path is too slow, which is the real defect

Where it sits

The architecture specification already has a control plane with four planes and an eight-level authority ladder. The Policy Platform is the product that makes one of those planes authorable, and it borrows the ladder rather than inventing a second one.



The contradiction we are not repeating

The reviewed approach describes a hot-path evaluation API — `POST /v1/policy-decisions` — and then, correctly, says the policy runtime must be local to the decision plane and consume a compiled bundle. Both cannot be true. Publishing an evaluation endpoint invites someone to call it over the network from the authorization path, and the first team under deadline pressure will. Here, runtime evaluation is an **in-process library call against a held bundle** and has no network endpoint at all. The APIs in §B10 exist for authoring, checking, simulating and explaining — every one of them off the payment path.

Namespaces and identity

Namespace is part of policy identity, not a folder. It determines the authority level, the owner, the approval chain and the precedence behaviour.

```
policy/
├─ regulatory/      authority: REGULATOR   owner: compliance
│  └─ us/{state}/   eu/ uk/ apac/ {country}/
├─ network/        authority: NETWORK     owner: network relations
│  └─ visa/ mastercard/ amex/ discover/ {debit_network}/
├─ psp/            authority: PSP         owner: institutional risk
│  └─ trust/ safety/ appetite/ prohibited_activity/
├─ risk/           authority: PSP         owner: risk
│  └─ fraud/ card_testing/ account_takeover/ merchant_risk/
├─ product/        authority: PSP         owner: product
│  └─ authentication/ credential/ routing/ recurring/
│     └─ marketplace/ card_present/ rails/
├─ platform/{id}/  authority: PLATFORM   owner: the platform
├─ isv/{id}/       authority: PLATFORM   owner: the ISV
├─ payfac/{id}/    authority: PAYFAC     owner: the PayFac
├─ franchise/{id}/ authority: PAYFAC*    owner: the franchisor
├─ merchant/{id}/  authority: MERCHANT   owner: the merchant
├─ store/{id}/     authority: SUBMERCHANT owner: merchant or franchisee
├─ operations/     authority: PSP         owner: operations
│  └─ incident/ maintenance/ emergency/

* franchise authority is configurable per franchise agreement and is
  recorded on the franchise relationship, because a franchisor's
  power over a franchisee varies by contract and by jurisdiction.
```

Risk is not its own ladder rung

The reviewed approach places risk in the precedence ladder with the caveat "assuming the risk policy is a hard safety control". That caveat is doing too much work. Here, risk *predictions* are context attributes that any policy may reference, and risk *policies* sit at PSP authority — the same level as institutional risk appetite. This removes the ambiguity about whether a given risk rule outranks a network mandate. It does not.

Capability catalogue

Policies act on capabilities with stable identifiers, never on service names or internal fields. This is the strongest idea in the reviewed approach and we did not have it — our action space was defined per decision point, which works at runtime and gives an author nothing to write against.

payment.authorization	payment.capture
payment.incremental	payment.refund
payment.reversal	payment.void
payment_methods.card	payment_methods.ach
payment_methods.rtp	payment_methods.wire
payment_methods.wallet	payment_methods.open_banking
authentication.3ds	authentication.exemption
authentication.delegated	authentication.passkey
authentication.challenge	
credential.store	credential.network_token
credential.acquirer_token	credential.account_updater
credential.fpan	
routing.processor	routing.network
routing.cross_border	routing.least_cost
routing.failover	
retry.attempt	retry.step_up
retry.credential_substitution	retry.processor_substitution
money.settlement	money.payout
money.reserve	money.fx_conversion
money.dcc	
card_present.offline	card_present.partial_approval
card_present.incremental	card_present.pin_bypass
recurring.cit	recurring.mit
recurring.installment	marketplace.split
marketplace.payout	agentic.scoped_token
Each entry carries	
capability_id	owner · description · requirements[]
effects_permitted[]	which verbs may target it
decision_point_binding	where it becomes an action-space constraint at runtime
status	ACTIVE DEPRECATED PLANNED

Attribute vocabulary

Governed, versioned and closed. An attribute not in the vocabulary fails compilation rather than evaluating to null at runtime — which is how `fraudScore`, `fraud_score` and `risk.score` come to coexist and mean different things.

Group	Attributes	Source
payment	amount · currency · channel · presence · capture_method · operation	Request
instrument	bin · bin_range · network · product_id · funding_type · commercial · prepaid · regulated_debit · issuer_id · issuer_country · token_type	Vault-time enrichment
geography	merchant_country · acquiring_country · issuer_country · billing_country · shipping_country · ip_country · residency_zone	Context assembly
merchant	id · country · mcc · legal_entity · risk_level · onboarding_state	Config snapshot
hierarchy	platform_id · isv_id · payfac_id · franchise_id · submerchant_id · store_id · terminal_id	Config snapshot
customer	id · country · tenure · risk_level	Context assembly
risk	p_fraud · p_dispute · p_false_decline · p_abuse · attack_state	Risk domain
authentication	status · method · exemption_claimed · liability_shift	Authentication domain
mandate	type · recurring · installment · cof · network_txn_id	Context assembly
device	id · type · managed · attestation	Device or SDK
agent	id · authorization · spending_limit · scope	Agentic flows

Governance rules

adding an attribute is a change with an owner and a review
renaming is a deprecation with a migration window, never in place
an attribute must have exactly one source domain
a policy referencing a deprecated attribute compiles with a
warning and fails after the window closes

Policy object and data model

```

Policy
├── policy_id · namespace · name · description
├── authority_level      derived from namespace, not declared
├── owner_type · owner_id
├── status · priority    priority orders WITHIN a level only
└── created_at · updated_at

PolicyVersion
├── policy_id · version · definition (DSL source) · checksum
├── valid_from · valid_to    when the rule applies in the world
├── recorded_at · superseded_at when we knew it
├── created_by · approved_by · approval_timestamp
└── state                  see §B11 lifecycle

PolicyScope
├── regions[] · countries[] · currencies[]
├── payment_methods[] · channels[] · networks[] · mccs[]
├── merchant_id · platform_id · isv_id · payfac_id
├── franchise_id · store_id · terminal_id
└── product

PolicyCondition
├── attribute · operator · value · data_type
└── grouping      AND / OR / NOT tree

PolicyEffect
├── effect_type      the algebra verb
├── capability_id    from the catalogue
├── value            for BOUND and PREFER
├── severity         HARD | ADVISORY
└── reason_code     canonical, merchant-visible

PolicyException · PolicyDelegation · PolicyBundle
PolicyDeployment · PolicyApproval · PolicyEvaluation
PolicyDecision · PolicySimulation · PolicyConflict

```

DSL

```

policy regulatory.eu.psd2.sca
  version 12
  valid_from 2026-01-01
  when
    geography.issuer_country in EEA
    and payment.channel = ECOMMERCE
    and payment.amount > 30 EUR
  then
    REQUIRE authentication.3ds
    unless authentication.exemption_claimed in [TRA, LOW_VALUE]
      reason_code SCA_REQUIRED
  severity HARD
  owner compliance.eu

```

Effects, algebra, precedence

Verb	Meaning	Composition behaviour
ALLOW	Permits a capability that would otherwise be unavailable	Additive within a level; cannot restore a higher-level DENY
DENY	Removes a capability for this context	Absorbing — no lower level can restore it
PROHIBIT	Removes a specific action within a capability	Absorbing
REQUIRE	Narrows the action space to the required subset	Intersecting; multiple REQUIREs intersect
BOUND	Constrains a numeric or enumerated range	Intersecting; tightest bound wins
PREFER	Orders what remains	Never adds or removes; lower level may reorder
DELEGATE	Transfers bounded authority downward	Explicit grant with scope, bounds and expiry

ActionSpace = Capability
 n Entitlement
 n (Allowed after DENY and PROHIBIT applied top-down)
 n (Required subset after REQUIRE)
 n Bounds
 ordered by PREFER

Precedence, descending
 REGULATOR → NETWORK → PSP → PLATFORM → PAYFAC → MERCHANT
 → SUBMERCHANT → LEARNED STRATEGY

Rules
 a lower level may never restore what a higher level denied
 a lower level may always further restrict
 numeric priority orders policies WITHIN a level; it has no
 effect across levels, and a policy declaring a priority
 intended to cross a level fails compilation
 DELEGATE is the only mechanism by which authority moves down,
 and it carries explicit bounds

Worked resolution

Regulator requires 3DS. Merchant disables 3DS. Network prohibits processor C for this transaction type. Merchant prohibits processor A. Risk, at PSP level, requires a challenge. Result: authentication is 3DS_CHALLENGE — the merchant's disable is inert and reported as inert — and the routing action space is [processor B, processor D]. The merchant is told their 3DS setting had no effect and why, rather than being left to discover it.

Exceptions

```
PolicyException
├─ exception_id · target_policy_id · target_version
├─ scope          narrower than the policy it excepts, always
├─ reason          free text, mandatory
├─ requested_by · approved_by · approval_chain
├─ effective_from · expires_at          expiry mandatory, no "never"
├─ authority_level cannot exceed the policy it excepts
└─ review_date
```

Hard rules

```
an exception may never except a REGULATOR or NETWORK policy –
  those require a policy change with the same approval chain,
  not an exception
an exception may never widen scope beyond its target
an exception without an expiry does not compile
expired exceptions lapse silently and the policy reasserts
exception count and age are reported to product monthly
```

Compilation and conflict detection



Conflict class	Detection	Resolution
Direct contradiction	Two policies, same capability, same scope, opposing verbs	Higher authority wins; lower reported inert; author warned at compile time
Unreachable policy	A policy whose conditions can never be satisfied given higher-level denials	Compiles with a warning; reported as inert
Scope overlap, same level	Two same-level policies with intersecting scope and different effects	Priority orders them; if priorities are equal, compilation fails
Bound contradiction	Two BOUNDS producing an empty range	Compilation fails and names both policies
Delegation breach	A delegated policy exceeding its grant's bounds	Compilation fails
Illegal exception	Exception against a regulator or network policy	Compilation fails
Vocabulary drift	Unknown or deprecated attribute	Fails, or warns within the migration window

Distribution and runtime

NORMAL PATH

compile → sign → regional registry → edge node in-process cache
propagation SLO p99 60 s · TTL-stamped · pre-warmed before
exposure switches on

EMERGENCY PATH

signed constraint delta → direct to edge
propagation SLO p99 5 s
scope: DENY and PROHIBIT and REQUIRE only – never PREFER,
never a bundle change, never an ALLOW
authority: REGULATOR, NETWORK or PSP only
dual control: two approvers, both recorded
always additive: may restrict, never permit
fail-closed: a node that cannot confirm application within its
deadline stops accepting traffic for the affected scope

RUNTIME

in-process evaluation over the held bundle
no network call · no database query · no cache miss path
p50 1.0 ms · p99 3.0 ms
output: ResolvedConstraintSet + ActionSpace + provenance

APIs

Five families, none of them on the payment path.

Family	Endpoints	Consumers
Management	<code>POST /v1/policies</code> · <code>GET /v1/policies/{id}</code> · <code>POST /v1/policies/{id}/versions</code> · <code>POST .../activate</code> · <code>POST .../rollback</code>	Compliance, network relations, risk, product, merchant console
Check	<code>POST /v1/policy/check</code>	Onboarding, dashboards, checkout UI, support tooling, sales
Simulate	<code>POST /v1/policy/simulations</code> · <code>GET /v1/policy/simulations/{id}</code>	Any author, before approval
Explain	<code>GET /v1/policy-decisions/{id}/explanation</code>	Support, operations, compliance, merchants
Deploy	<code>POST /v1/policy-bundles</code> · <code>/validate</code> · <code>/shadow</code> · <code>/canary</code> · <code>/deploy</code> · <code>/rollback</code>	Platform stewards

Check — the one we were missing

```
POST /v1/policy/check
{ "subject": { "merchant_id": "mrc_123" },
  "capability": "payment_methods.ach",
  "context": { "country": "US", "mcc": "5812", "amount": 10000 } }

→ { "allowed": false,
    "reason_code": "CAPABILITY_PROHIBITED_BY_PLATFORM",
    "policy": "platform.plt_77.ach_restrictions",
    "policy_version": 4,
    "authority": "PLATFORM",
    "merchant_can_resolve": false,
    "resolution_path": "Contact platform operator" }
```

This single endpoint removes a class of support ticket and a class of sales promise that cannot be kept. It powers onboarding validation, merchant dashboards, checkout method presentment and the solutions team's pre-sale answer — all from the same compiled bundle the payment path uses, so they cannot diverge.

Simulate

```
POST /v1/policy/simulations
{ "base_bundle": "pb_987",
  "proposed": [ { "policy": "merchant.mrc_123.international",
                  "change": "activate", "version": 5 } ],
  "sample": "last_30_days" }

→ { "transactions_evaluated": 12843091,
    "decisions_changed": 48291,
    "newly_permitted": 42100, "newly_denied": 6191,
    "by_reason_code": { ... },
    "affected_merchants": 1,
    "conflicts_introduced": 0,
    "estimated_authorization_impact": { "point": "+0.18%",
                                         "ci": ["+0.11%", "+0.25%"] } }
```

An important distinction in what simulation can promise

The count of changed decisions is **exact**. Policy is deterministic, so replaying thirty days against a proposed bundle tells you precisely which transactions would have been decided differently — no estimation involved. The *outcome* impact is a different matter: whether a newly permitted transaction would have been approved is a counterfactual, and it carries an interval. Reporting both under one heading would let an exact number lend false authority to an estimated one, so they are separated in the response and must be separated in the UI.

Explain

```
GET /v1/policy-decisions/{decision_id}/explanation

→ { "decision": "DENY payment.authorization",
    "why": [ "Issuer country is on the prohibited list for this merchant's acquiring entity",
             "No exception is in force for this merchant" ],
```

```
"policy_path": [ "regulatory.us.ofac_adjacent_geographies",  
  "psp.trust.prohibited_geographies" ],  
"authority": "REGULATOR",  
"merchant_visible_reason": "GEOGRAPHY_NOT_SUPPORTED",  
"overrides_considered": [],  
"bundle": "pb_987@41",  
"bitemporal": { "valid_at": "...", "known_as_of": "..." }
```

Lifecycle and failure modes

DRAFT → VALIDATING → SIMULATING → APPROVAL_REQUIRED → APPROVED
→ SHADOW → CANARY → ACTIVE → SUPERSEDED

alternates: REJECTED · ROLLED_BACK · EXPIRED · INERT

SHADOW and CANARY are skipped only for emergency prohibitions, where the whole point is speed and the change is additive and restrictive. Every emergency policy carries a 30-day expiry and a mandatory post-review that either normalises it into a standard policy or withdraws it.

Failure	Behaviour
Bundle expired, control plane unreachable	Node degrades to the most restrictive held constraint set and reports degraded. There is no configuration in which a node evaluates indefinitely on an artefact it cannot refresh
Bundle signature invalid	Rejected; prior version retained; alert raised
Evaluation exceeds budget	HARD_FAIL. Constraint binding has no fallback — proceeding under an unknown constraint set is worse than declining
Emergency delta unconfirmed on a node	That node stops accepting traffic for the affected scope
Conflict discovered post-deployment	Higher authority wins at runtime; conflict raised as a defect against the compiler, since it should have been caught
Policy references an attribute absent from context	Compilation would have caught it. At runtime this is a platform defect and the transaction fails closed

Why regulatory blocking first

Reason	Detail
Deterministic	No optimisation surface, no model, no experimentation. Right or wrong is decidable
Highest authority	Exercises the top of the precedence ladder, where absorbing verbs and exception prohibitions matter most
Real and current	These rules exist today and are enforced by code and by hope
Exercises everything	Namespace, scope, vocabulary, effects, compilation, bitemporality, emergency path, explanation, conflict detection
Low blast radius	A blocking policy that misfires declines transactions. Unpleasant, visible immediately, and reversible in seconds — unlike a permissive policy that misfires
Audit value from day one	"Prove this rule was in force on that date" becomes answerable with evidence

The four policy shapes

1 · Block a specific BIN

```
policy regulatory.us.prohibited_issuers
version 3
valid_from 2026-10-01
when
  instrument.bin in ["411111", "492345", "545454"]
then
  DENY payment.authorization
  reason_code ISSUER_NOT_PERMITTED
severity HARD
owner compliance.us
```

2 · Block a BIN range

```
policy regulatory.us.prohibited_issuer_ranges
version 7
valid_from 2026-10-01
when
  instrument.bin_range overlaps [
    "4111110000000000" .. "4111119999999999",
    "5454540000000000" .. "5454549999999999"
  ]
then
  DENY payment.authorization
  reason_code ISSUER_NOT_PERMITTED
severity HARD
```

Ranges are expressed as full-length account-number intervals, not six or eight digit prefixes. Scheme range files are published as intervals, prefix lengths vary by network and have changed over time, and an interval representation avoids re-encoding every time a network extends a range.

3 · Restrict a currency within a geography

```
policy regulatory.in.currency_restriction
version 2
valid_from 2026-11-15
when
  geography.merchant_country = "IN"
  and payment.currency != "INR"
then
  DENY payment.authorization
  reason_code CURRENCY_NOT_PERMITTED_IN_JURISDICTION
severity HARD
owner compliance.apac
```

4 · Block a geography, with the ambiguity resolved

```
policy psp.trust.prohibited_geographies
version 11
valid_from 2026-09-01
when
  geography.issuer_country in PROHIBITED_LIST
  or geography.billing_country in PROHIBITED_LIST
  or geography.ip_country in PROHIBITED_LIST
then
  DENY payment.authorization
  reason_code GEOGRAPHY_NOT_SUPPORTED
severity HARD
owner risk.institutional
```

Which country the rule means

"Block country X" is four different rules. Issuer country is where the card was issued. Billing country is what the cardholder typed. IP country is where they appear to be. Merchant country is where we acquire. They disagree constantly and legitimately — a US-issued card used by a traveller in Europe against a UK merchant produces three different answers. A policy that does not name which country it means will be interpreted differently by whoever implements it, and the vocabulary in §B4 exists so it cannot be written ambiguously. The DSL has no bare `country` attribute.

BIN matching mechanics

WHERE THE BIN COMES FROM

instrument.bin and instrument.bin_range arrive with the credential, resolved at vault time from the scheme range files (spec §08). There is NO runtime BIN lookup – that is what keeps constraint evaluation inside 3 ms.

MATCHING STRUCTURE

ranges compile into an interval tree keyed by numeric account range. Lookup is $O(\log n)$ over a few hundred thousand intervals, in memory, with no allocation on the hot path.

the compiler merges overlapping intervals, detects containment, and reports a range wholly contained in another as inert.

FRESHNESS INTERACTION

the BIN attributes carry as_of and freshness_class (spec §08). A blocking policy must not be defeated by a stale attribute, so for DENY effects at REGULATOR or PSP authority the platform applies the conservative reading:

attribute fresh	→ evaluate normally
attribute stale	→ evaluate normally, flag
attribute missing	→ DENY, with reason_code ISSUER_ATTRIBUTES_UNAVAILABLE

this is the opposite of the optimisation posture in §12 of the specification, where missing attributes suppress optimisation rather than blocking. Blocking policies fail closed; optimising decisions fail open. Both are deliberate.

Stored credentials and mandates

The operational question a BIN block raises and that a rules engine does not answer: what happens to credentials already in the vault and to recurring mandates already agreed.

Object	On a new blocking policy
New authorizations	Denied from the moment the bundle is in force
Stored credentials matching the block	Marked <code>POLICY_SUSPENDED</code> , not deleted. The block may be withdrawn, and deletion is irreversible
Network tokens on those credentials	Suspended, not deleted — deletion forfeits the token and the provisioning relationship
Account updater	Suspended for suspended credentials; no point refreshing something unusable
Recurring mandates	Flagged; the merchant is notified with the affected customer count, not the identities, unless entitled
In-flight authorizations	Complete under the constraint set bound at their start. Ownership of a decision never changes mid-transaction
Captures against prior authorizations	Permitted. Blocking authorization does not strand funds already authorised — a separate policy is required to block capture, and it should be rare
Refunds	Always permitted. A block must never trap a cardholder's money
Disputes and chargebacks	Unaffected. Scheme obligations survive our policy

The rule underneath that table

A block stops new exposure. It does not unwind existing obligations, strand settled funds, or prevent money returning to a cardholder. Any policy that would do one of those requires a separate, explicitly approved policy targeting that capability — and the capability catalogue separates `payment.authorization` , `payment.capture` and `payment.refund` precisely so this cannot happen by accident.

Boundary with screening

	Screening	Policy
Question	Is this party on a list	Is this transaction permitted
Subject	Names, entities, addresses	Instrument, geography, currency, capability
Source	Sanctions and watchlist data	Authored rules
Position	Before constraint binding, parallel	Constraint binding
Disposition	HARD_FAIL, no fallback	HARD_FAIL, no fallback
Reason code	SCREENING_BLOCKED	POLICY_BLOCKED with a specific sub-code
Explainability	Deliberately opaque to the merchant	Explained, with the authority level named

Both run, both can decline, and neither substitutes for the other. A country-level prohibition is policy; a person on a sanctions list is screening. Conflating them produces either a policy engine holding sanctions data it should not, or a screening module holding business rules nobody can audit.

Acceptance tests

#	Test	Expected
1	Author a BIN block, simulate against 30 days	Exact count of transactions that would have been declined, by merchant and reason code
2	Activate it; submit a matching transaction	Declined, <code>POLICY_BLOCKED / ISSUER_NOT_PERMITTED</code> , provenance names policy and version
3	Submit a transaction one digit outside the range	Approved. Boundary conditions are the failure mode for interval matching
4	Merchant writes a policy permitting the blocked BIN	Compiles, reported inert, merchant told why
5	Someone requests an exception against the regulatory policy	Compilation fails — exceptions may not target regulator-authority policies
6	Vault a credential in a blocked range	Credential stored, marked <code>POLICY_SUSPENDED</code> , tokens suspended not deleted
7	Refund a prior settled transaction on a now-blocked BIN	Permitted
8	Instrument attributes missing on a blocked-range candidate	Denied, <code>ISSUER_ATTRIBUTES_UNAVAILABLE</code>
9	Emergency-propagate a country block	Applied globally within 5 s p99; unconfirmed nodes stop accepting traffic in scope
10	Correct the policy retroactively; query affected transactions	Exact list of transactions decided under the superseded record, via bitemporal query
11	Roll back the bundle	Pointer change; effective within propagation SLO; no data migration
12	Support agent opens the explanation for a declined transaction	Under two minutes, self-serve, no engineer

Disposition of the reviewed approach

Proposal	Disposition
Policy namespaces as part of identity	Adopted. Extended so authority level is derived from namespace rather than declared, which removes a whole class of privilege-escalation error
Capability model as the target of effects	Adopted. The strongest idea in the review and a genuine gap — our action space was defined per decision point, which works at runtime and gives an author nothing to write against
Governed attribute namespace	Adopted. The observation about <code>fraudScore</code> versus <code>risk.score</code> is exactly right; unknown attributes now fail compilation
"Can I?" check API	Adopted. We did not have it. It removes a class of support ticket and a class of unkeepable sales promise
Conflict detection with named resolution	Adopted and expanded to seven conflict classes, three of which fail compilation outright
Simulation API	Adopted, modified. Decision changes are exact because policy is deterministic; outcome impact is a counterfactual with an interval. The response separates them so an exact number cannot lend authority to an estimated one
Explainability API	Adopted , with the authority level and bitemporal coordinates added
Exceptions as first-class objects	Adopted , with mandatory expiry and a prohibition on excepting regulator or network policies
Policy lifecycle with shadow and canary	Adopted , with emergency prohibitions explicitly exempt and time-limited
Deployment API family	Adopted
<code>POST /v1/policy-decisions</code> as a hot-path API	Declined. It contradicts the same document's correct conclusion that the runtime must be local and bundle-driven. Publishing the endpoint guarantees someone calls it from the payment path. Runtime evaluation is an in-process library call with no network endpoint
Thirteen effects normalised to five	Declined in favour of ours. Seven verbs with defined composition, including DELEGATE — which the proposed set omits and which PayFac and platform models require
Risk as a precedence level	Declined. Risk predictions are context attributes; risk policies sit at PSP authority. The proposal's "assuming the risk policy is a hard safety control" hides the ambiguity rather than resolving it
Numeric priority alongside hierarchical precedence	Modified. Priority orders within a level only. A policy declaring a priority intended to cross a level fails compilation
Bitemporality	Not in the proposal; retained from ours. The proposed <code>PolicyVersion</code> has validity time but no knowledge time, so "what did we believe was in force" is unanswerable — and that is the question in a regulatory finding
Emergency propagation	Not in the proposal; retained from ours. Shadow-then-canary is right for normal changes and useless for a sanctions addition
Data residency	Not in the proposal; added. Bundles compile and distribute per residency zone; evaluation never leaves it